



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Operating Systems and Systems Programming – Project Implementation Guidelines

All sections of the course shall follow a common project framework to ensure uniformity in project scope, implementation requirements, and evaluation.

1. Implementation Platform

All student projects shall be implemented and demonstrated on a Linux environment, preferably Ubuntu.

Students should use C/C++ with Linux/POSIX system programming interfaces such as:

- Process creation and management
- Linux system calls
- Pipes and FIFOs
- Signals
- POSIX threads
- Mutexes, condition variables, and semaphores
- File I/O and file descriptors
- Memory management and memory-mapped files
- /proc and related Linux interfaces
- GDB, strace, and other appropriate debugging/analysis tools

Students are NOT required to modify the Linux kernel. Projects should primarily be implemented as user-space Linux/system-programming applications.

2. Project Selection

Faculty shall allocate projects from the approved project-title list provided for the course.

The project titles should be retained as far as possible so that all 13 sections follow a common project structure.

Faculty may guide students in selecting an appropriate project scope based on student ability, but should ensure that the minimum functionality and learning outcomes are achieved.

3. Project Scope

Each project must contain:

1. Problem definition

2. OS/system-programming concepts involved
3. System design
4. Linux/POSIX APIs used
5. Working implementation
6. Test cases and experimental results
7. Performance or behavior analysis
8. Documentation
9. Final demonstration and viva

Projects should demonstrate actual implementation rather than only theoretical study.

4. Faculty Responsibility

Each section instructor shall:

- Allocate projects to student teams
- Ensure that projects are appropriately challenging
- Monitor project progress periodically
- Review the design and implementation
- Ensure that students use Linux/POSIX concepts appropriately
- Verify that the submitted implementation is functional
- Conduct project demonstrations and viva
- Maintain evidence of student contribution and project progress

5. Common Evaluation

All sections should follow a common evaluation structure.

A suggested distribution is:

- Problem definition and objectives – 10%
- OS concepts and CO mapping – 15%
- System design – 15%
- Implementation – 30%
- Testing and results – 15%
- Documentation – 5%
- Demonstration and viva – 10%

6. Important Restriction

The projects should not become generic application-development projects.

For example, a project should not simply be a GUI, web application, database application, or CRUD system.

The core implementation must demonstrate Operating Systems and Systems Programming concepts through Linux/POSIX mechanisms.

7. Final Deliverables

Each team shall submit:

- Source code
- Makefile/build instructions
- README
- Project report
- Test cases and results
- Screenshots/output evidence
- Final presentation
- Working demonstration

Each student should be able to explain the implementation, system calls/APIs used, design decisions, and OS concepts involved.

Appendix A: Student Project Titles

1. Concurrent Web Server for University Resource Portal

Scenario

A university provides a web portal through which students access notices, course materials, examination schedules, and other academic resources. During peak hours, hundreds of students may access the portal simultaneously, causing delays and reduced server performance.

Problem

The university needs a concurrent server that can handle multiple client requests efficiently while safely managing shared resources and delivering files with minimal overhead.

Student Task

Build a multithreaded web server in C that:

- Accepts multiple TCP client connections simultaneously
- Creates or manages a pool of worker threads to process client requests
- Maintains a shared connection queue for incoming clients
- Uses mutexes and semaphores to synchronize access to shared resources
- Serves static HTML, text, and other files to clients
- Uses memory-mapped files (`mmap()`) for efficient file access
- Maintains server activity and request logs
- Uses signals to support server control and configuration reload
- Monitors active threads and server resource usage

2. CPU Scheduling Simulation System

Scenario

An operating-system laboratory wants to demonstrate how a CPU manages multiple processes competing for processor time. Students need to observe how different scheduling algorithms affect waiting time, turnaround time, and CPU utilization.

Problem

The laboratory needs a simulator that can create multiple processes, communicate process information between components, simulate CPU interrupts, and evaluate different scheduling strategies.

Student Task

Build a CPU scheduling simulator in C that:

- Creates and maintains multiple simulated processes
- Implements FCFS, SJF, Priority, and Round Robin scheduling
- Uses pipes to transfer process information between simulator components
- Uses named pipes (FIFOs) for communication between independent modules
- Uses signals to simulate timer interrupts and time-slice expiration
- Dynamically allocates memory for process-control information
- Tracks process states such as ready, running, waiting, and terminated
- Calculates waiting time, turnaround time, and response time
- Displays scheduling results and performance statistics

3. Flight Simulator Control and Monitoring System

Scenario

A flight simulation environment contains several independent components such as altitude sensors, speed sensors, engine monitors, and navigation systems. These components continuously generate information that must be delivered to a central flight controller.

Problem

The simulator needs a reliable inter-component communication mechanism that can transfer sensor information and respond immediately to critical events such as engine failure or emergency shutdown.

Student Task

Build a flight simulator control system in C that:

- Creates independent processes representing different aircraft components
- Generates simulated altitude, speed, engine, and navigation data
- Uses pipes for communication between simulator components
- Uses named pipes (FIFOs) for communication between independent processes

- Uses signals to simulate emergency events and hardware interrupts
- Detects and responds to simulated failures
- Dynamically allocates memory for sensor and flight-state information
- Synchronizes access to shared simulation data
- Maintains logs of sensor events and emergency conditions
- Displays the current simulated aircraft state

4. Multi-Agent AI Execution and Resource Management System

Scenario

An AI development environment uses multiple autonomous agents to perform different tasks such as generating code, executing programs, reviewing code, and testing applications. These agents need to communicate and operate on a shared workspace.

Problem

The organization needs a controlled execution environment that allows multiple AI agents to work together while preventing unauthorized file access and excessive CPU or memory consumption.

Student Task

Build a multi-agent AI execution system in C that:

- Creates separate processes representing different AI agents
- Uses named pipes (FIFOs) for communication between agents
- Transfers structured task and execution information between agents
- Provides a restricted workspace for agent operations
- Validates file access before allowing an agent to perform an operation
- Monitors CPU and memory consumption of agents
- Uses signals to stop or control agents exceeding resource limits
- Maintains synchronization between agents working on shared files
- Automatically records agent activities and execution results
- Maintains backup copies of modified workspace files

5. University Laboratory Management Server

Scenario

A university computer laboratory contains a large number of systems accessed by students simultaneously. Students may request system resources, submit programs, access files, and perform laboratory activities at the same time.

Problem

The laboratory needs a centralized management system that can handle multiple student requests concurrently while controlling access to limited resources.

Student Task

Build a university laboratory management server in C that:

- Accepts multiple client requests simultaneously
- Uses worker threads to process student requests
- Maintains a shared request queue
- Uses mutexes to protect shared data structures
- Uses semaphores to control access to limited laboratory resources
- Maintains student requests and resource allocation information
- Uses signals for server administration and controlled shutdown
- Monitors active processes and threads
- Maintains server activity logs
- Detects and reports resource usage and availability

6. Database Client-Server Management System

Scenario

A university maintains a centralized student database containing student information, marks, attendance, and course records. Multiple faculty members and administrative users may access the database simultaneously.

Problem

The organization needs a client-server system capable of handling concurrent database requests while preventing corruption of shared data.

Student Task

Build a database client-server system in C that:

- Accepts requests from multiple clients
- Creates worker threads to process concurrent requests
- Maintains a shared request and response queue
- Uses mutexes to protect shared database structures
- Uses semaphores to control concurrent access to resources
- Supports operations such as insert, search, update, and delete
- Uses memory mapping for efficient access to frequently used data
- Maintains transaction and server activity logs
- Uses signals for configuration reload and controlled shutdown
- Monitors active client connections and server resources

7. Linux Process Resource Monitoring System

Scenario

A system administrator manages a Linux server running hundreds of processes. Some processes may consume excessive CPU or memory, become unresponsive, or continue running indefinitely.

Problem

The administrator needs a monitoring system that continuously observes processes and automatically responds when resource usage exceeds predefined limits.

Student Task

Build a Linux process monitoring system in C that:

- Monitors running processes and their states
- Collects CPU and memory usage information
- Reads process information from the Linux `/proc` filesystem
- Detects processes exceeding predefined resource limits
- Uses signals to stop or terminate problematic processes
- Maintains process activity and resource-usage logs
- Tracks process creation and termination
- Generates warnings when resource thresholds are exceeded
- Displays current system and process information
- Provides an administrator-controlled mechanism for starting and stopping monitoring

8. High-Concurrency File Distribution Server

Scenario

A university frequently distributes large lecture videos, PDFs, laboratory manuals, and software packages to thousands of students. Traditional file-copy operations can consume significant CPU and I/O resources when many students request the same files simultaneously.

Problem

The university needs a high-performance file distribution server that can efficiently serve large files to many clients while minimizing unnecessary memory and I/O operations.

Student Task

Build a file distribution server in C that:

- Accepts multiple client connections concurrently
- Uses worker threads to process file requests
- Maintains a synchronized request queue
- Uses mutexes and semaphores for shared resource protection
- Uses `mmap()` to access large files efficiently
- Transfers requested files to clients through TCP sockets
- Maintains download and client activity logs
- Monitors active connections and resource consumption
- Uses signals for server control and shutdown
- Handles multiple simultaneous requests for the same file

9. Alarm Clock System

Scenario

A Linux-based system is required to provide an alarm facility that allows users to schedule an alarm for a specific time. The system should continue performing other activities while waiting for the alarm time.

Problem

The system needs a mechanism to suspend a process while waiting and automatically notify it when the specified time is reached.

Student Task

Build an alarm clock system in C that:

- Accepts an alarm time or delay from the user
- Creates a process to manage the alarm
- Uses timers or signals to generate the alarm event
- Suspends the process while waiting for the alarm
- Handles the alarm signal using an appropriate signal handler
- Displays an alarm notification when the specified time is reached
- Allows the user to cancel an active alarm
- Handles multiple alarm requests

10. Priority Scheduling System

Scenario

A computing system has multiple processes competing for CPU time. Some processes are more important than others and must receive CPU time before lower-priority processes.

Problem

The system needs a scheduler that selects processes according to their priority while maintaining correct process execution and completion.

Student Task

Build a priority scheduling system in C that:

- Creates multiple processes with different priorities
- Maintains process information such as PID, priority, arrival time, and burst time
- Selects the highest-priority process for execution
- Handles processes with equal priorities
- Uses process creation and synchronization mechanisms
- Allows processes to execute and terminate correctly
- Calculates waiting time and turnaround time
- Displays the scheduling order and performance results

11. Process Termination Message System

Scenario

A system administrator needs to know when child processes created by an application finish their execution. Simply terminating a child process is not sufficient; the parent should also receive information about how and why the child terminated.

Problem

The system needs a mechanism for detecting child-process termination and displaying meaningful termination information to the parent process.

Student Task

Build a process termination monitoring system in C that:

- Creates multiple child processes
- Allows each child to perform a different task
- Detects when a child process terminates
- Uses `wait()` or `waitpid()` to collect child termination status
- Identifies normally terminated and abnormally terminated processes
- Displays the PID of the terminated process
- Displays the child's exit status
- Reports whether a process was terminated by a signal
- Prevents terminated child processes from becoming zombies

12. Command Argument Passing System

Scenario

A command-line application needs to execute different programs while passing user-provided arguments to them. The system must correctly transfer the command and its arguments from the parent application to the newly executed program.

Problem

The system needs to demonstrate how processes receive command-line arguments and how a new program can be executed with those arguments.

Student Task

Build an argument-passing system in C that:

- Accepts a command and multiple arguments from the user
- Creates a child process using `fork()`
- Passes arguments to the child process
- Uses an appropriate `exec()` function to execute the requested program
- Displays the arguments received by the new program

- Handles invalid commands and arguments
- Allows different commands to be executed with different argument lists
- Uses `wait()` or `waitpid()` to synchronize the parent and child

13. System Call Demonstration and Monitoring System

Scenario

A systems programming laboratory wants students to understand how applications request services from the Linux kernel. Programs perform operations such as creating files, reading data, writing data, and creating processes through system calls.

Problem

The laboratory needs a practical system that demonstrates system calls and allows students to observe the interaction between a user program and the operating system.

Student Task

Build a system-call demonstration program in C that:

- Creates and manipulates files using system calls
- Uses `open()`, `read()`, `write()`, and `close()`
- Creates processes using `fork()`
- Executes another program using `exec()`
- Synchronizes processes using `wait()` or `waitpid()`
- Checks and handles system-call return values
- Handles errors using `errno`
- Uses `strace` to trace system calls generated by the program
- Identifies the transition between user-space application code and kernel services

14. Executable Protection System

Scenario

A Linux system administrator wants to prevent important executable programs from being modified while they are being executed. Allowing another process to overwrite an executable could result in corrupted or inconsistent program execution.

Problem

The system needs to demonstrate how Linux protects executable files from being modified while they are in use.

Student Task

Build an executable protection demonstration in C that:

- Creates and executes a sample executable
- Attempts to modify the executable while it is running
- Observes the operating system's response
- Demonstrates the behavior of write operations on an active executable
- Uses system calls to open and modify files
- Checks and reports errors returned by the kernel
- Uses `errno` to identify the reason for failure
- Uses `strace` to observe the relevant system calls
- Explains how the operating system protects executable files

15. Linux System Debugging Tool

Scenario

A systems programmer develops a C application that occasionally crashes, hangs, or behaves unexpectedly. The programmer needs to observe the application's execution and identify the source of the problem.

Problem

The development environment needs a debugging system that can inspect processes, system calls, memory-related problems, and execution behavior.

Student Task

Build a Linux debugging environment in C that:

- Creates a test program containing controlled execution errors
- Observes process execution and termination
- Uses debugging tools to identify program failures
- Uses `strace` to monitor system calls
- Examines process information using Linux monitoring facilities
- Detects invalid system-call usage and error conditions
- Checks return values and `errno`
- Identifies abnormal process termination
- Documents the cause and location of detected problems

16. Kernel Monitoring System

Scenario

A Linux administrator needs to observe the internal activity of the operating system, including running processes, CPU usage, memory consumption, and system activity.

Problem

The administrator needs a monitoring system that collects information about the operating system and presents useful information about its current state.

Student Task

Build a Linux kernel monitoring system in C that:

- Monitors currently running processes
- Displays process IDs and parent process IDs
- Tracks process states
- Monitors CPU and memory usage
- Reads process information from `/proc`
- Displays information about system resources
- Detects process creation and termination
- Records important system events
- Provides a continuously updated monitoring display

17. Linux Process Creation, Execution and Termination System

Scenario

A university operating-systems laboratory wants a single application that demonstrates the complete lifecycle of a Linux process, from creation through program execution to termination.

Problem

Students need to understand how a parent process creates a child, how the child can replace its program using `exec()`, and how the parent detects the child's termination.

Student Task

Build a Linux process lifecycle system in C that:

- Creates child processes using `fork()`
- Displays parent and child PIDs
- Displays parent-child relationships using PPID
- Executes a new program from the child using `exec()`
- Allows the parent to continue or wait for the child
- Uses `wait()` or `waitpid()` to collect termination status
- Displays the child's exit status
- Handles abnormal child termination
- Demonstrates zombie and orphan process behavior
- Displays the complete process lifecycle from creation to termination

18. Linux Process Monitoring and Control System

Scenario

A Linux server may run hundreds of processes simultaneously. Some processes may consume excessive CPU or memory, become unresponsive, or need to be stopped by an administrator.

Problem

The administrator needs a process management system that can monitor running processes and provide controlled actions such as stopping, continuing, or terminating selected processes.

Student Task

Build a Linux process monitoring and control system in C that:

- Lists currently running processes
- Displays PID, PPID, process state, CPU usage, and memory usage
- Reads process information from `/proc`
- Allows the administrator to select a process
- Sends signals to selected processes
- Supports stopping, continuing, and terminating processes
- Detects processes that exceed predefined resource limits
- Records process-control activities in a log
- Monitors process creation and termination
- Provides a continuously updated process-monitoring interface

19. Linux Service and Process Supervisor

Scenario

A production Linux server runs many background services such as web servers, database services, monitoring agents, and scheduled tasks. If a service crashes, administrators need the system to detect the failure and restart it automatically.

Problem

The organization needs a lightweight service-management system that can start, stop, monitor, restart, and control background processes while maintaining their execution status.

Student Task

Build a Linux service and process supervisor in C that:

Reads service configurations from a configuration file

Creates and starts service processes

Uses `fork()` and `exec()` to launch services

Monitors the state of child processes

Detects unexpected process termination

- Automatically restarts failed services
- Uses waitpid() to collect process termination information
- Uses signals to stop, continue, or terminate services
- Maintains service status and activity logs
- Supports controlled shutdown of all managed services
- Displays the current state of all registered services

20. Asynchronous File Processing Server

Scenario

A cloud storage company receives thousands of file operations from clients. Each request may involve opening files, reading data, writing data, and returning results to the client. Processing every request synchronously can reduce server performance.

Problem

The organization needs an asynchronous I/O server capable of handling a large number of file and network operations without creating a separate blocking thread for every request.

Student Task

Build an asynchronous file-processing server in C that:

- Accepts multiple client requests
- Processes file operations asynchronously
- Maintains a queue of pending I/O requests
- Handles multiple file-read and file-write operations
- Uses asynchronous I/O mechanisms
- Maintains request and completion queues
- Synchronizes access to shared request data
- Supports concurrent client connections
- Records request and completion times
- Monitors server performance
- Reports throughput and request latency

Course Coordinator

HoD